# Lecture 6

# Process-to-Process Delivery: UDP, TCP

# Outline

- **6-1 PROCESS-TO-PROCESS DELIVERY**
- **6-2 USER DATAGRAM PROTOCOL (UDP)**
- **6-3 TCP**

# 6-1 PROCESS-TO-PROCESS DELIVERY

*The transport layer is responsible for process-to-process delivery—the delivery of a packet, part of a message, from one process to another. Two processes communicate in a client/server relationship, as we will see later.*

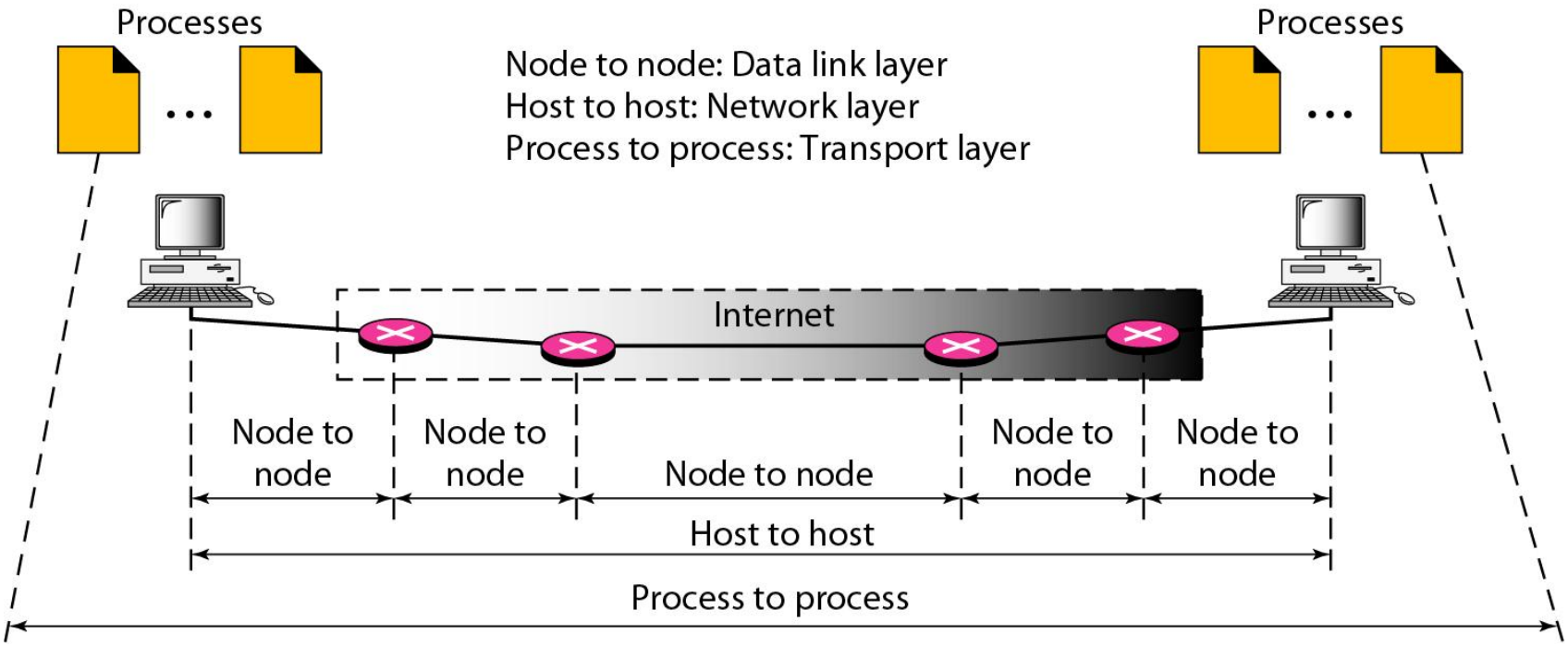
***Topics discussed in this section:***

**Client/Server Paradigm**
**Multiplexing and Demultiplexing**
**Connectionless Versus Connection-Oriented Service**
**Reliable Versus Unreliable**
**Three Protocols**

*Note*

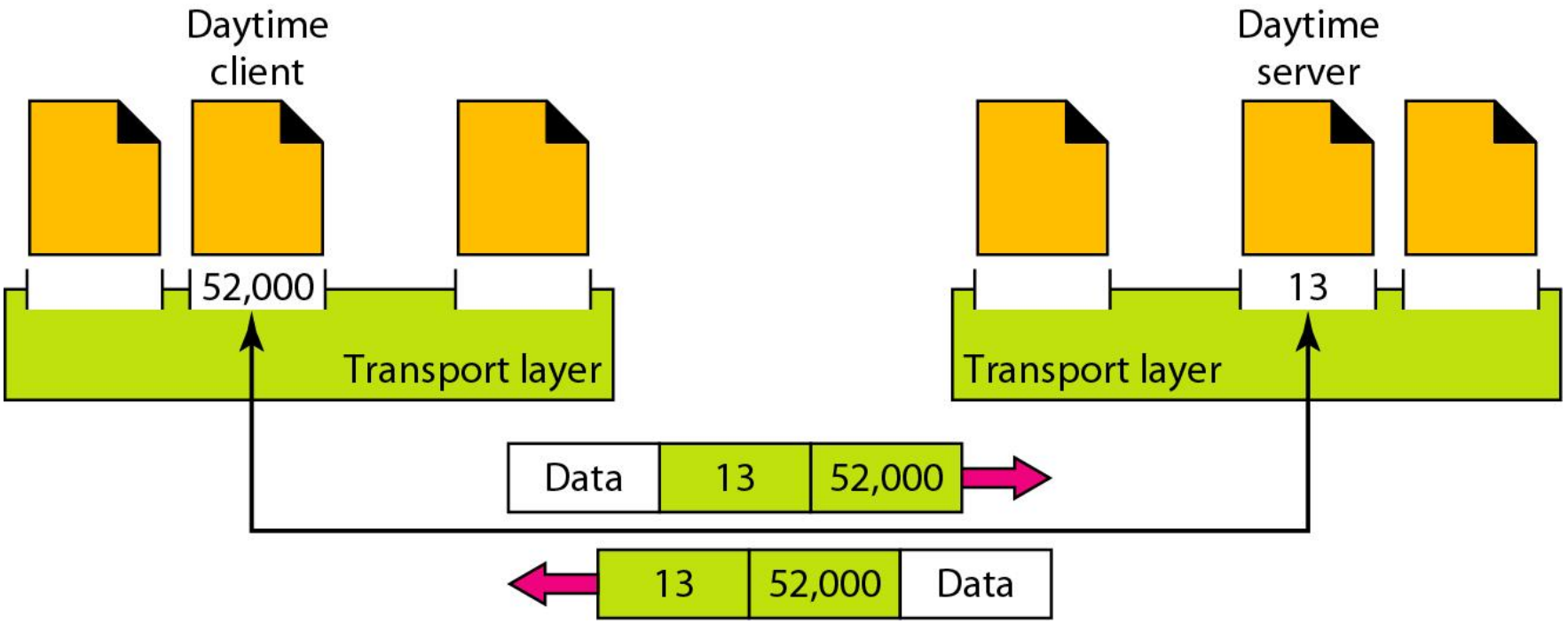
**The transport layer is responsible for process-to-process delivery.**

## Figure 6.1 *Types of data deliveries*

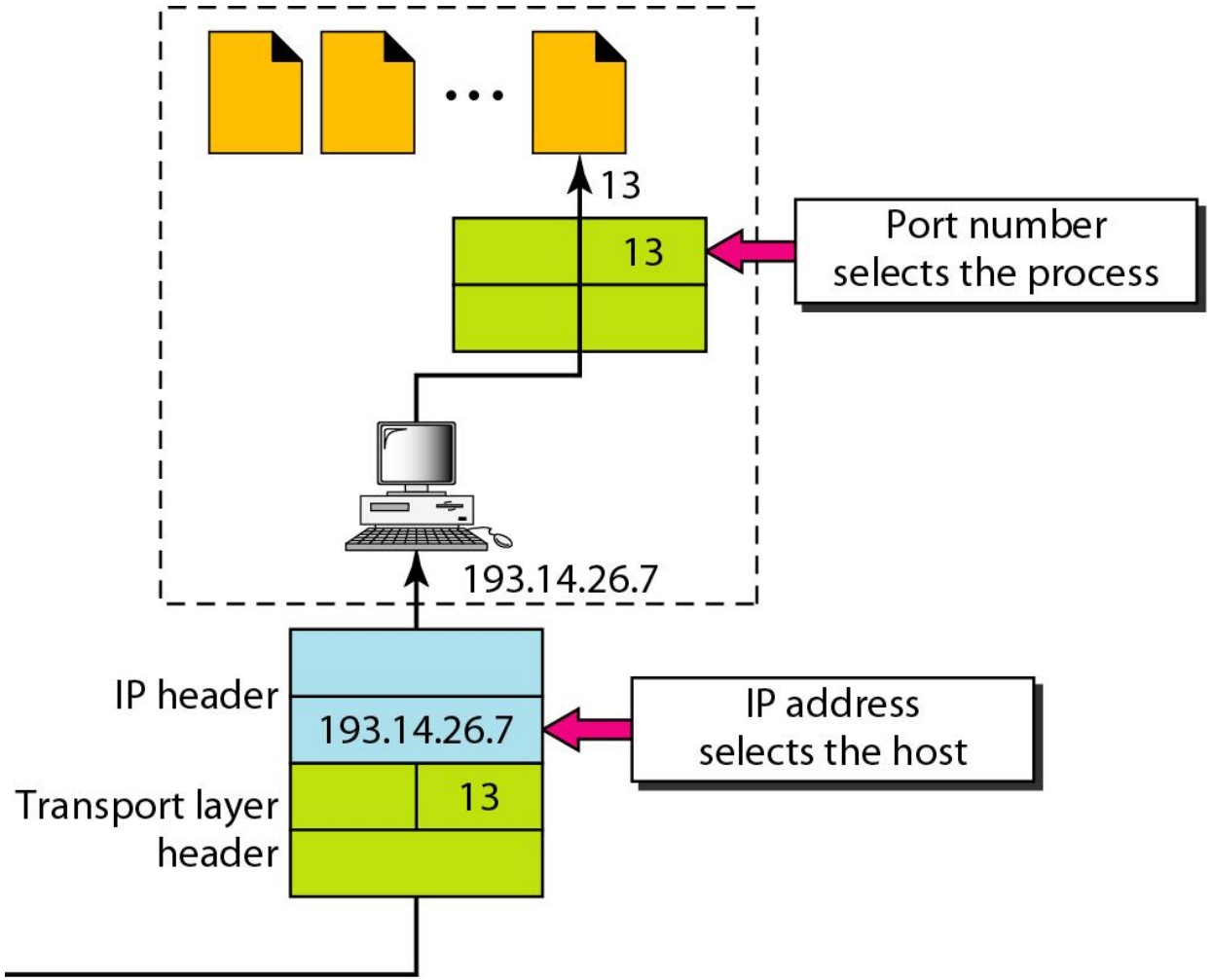


# Figure 6.2 *Port numbers*

The client uses an temporary port number 52,000 to identify itself, the Daytime server process must use the well-known (permanent) port number 13.



## Figure 6.3 *IP addresses versus port numbers*

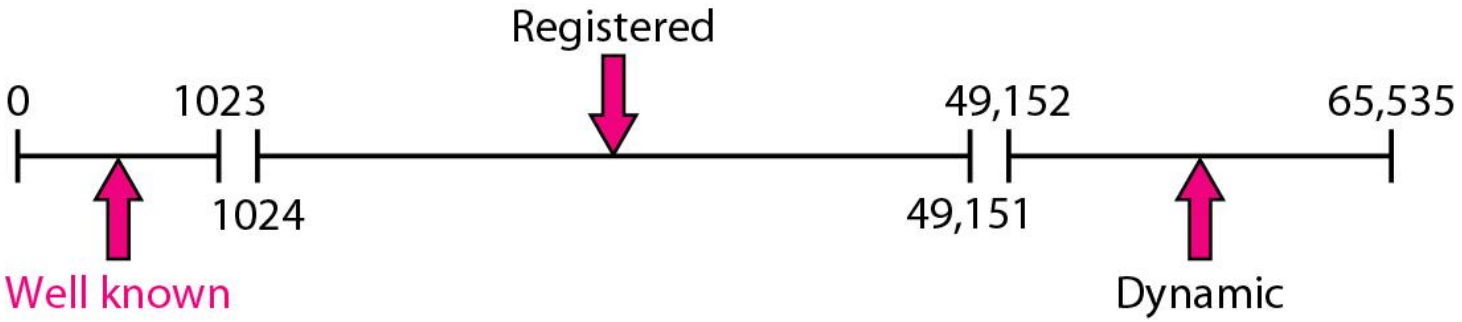



The destination IP address defines the host among the different hosts in the world. After the host has been selected, the port number defines one of the processes on this particular host.

# Figure 6.4 *IANA ranges*

**IANA: Internet Assigned Number Authority**

**Well-known ports: Assigned and controlled by IANA**
**Registered ports: They can only be registered with IANA to prevent duplication**
**Dynamic ports: They can be used by any process.**



# Figure 6.5 *Socket address*

The combination of an IP address and a port number is called a socket address.

The client socket address defines the client process uniquely.
The server socket address defines the server process uniquely.

A transport layer protocol needs a pair of socket address: The client socket address and The server socket address.

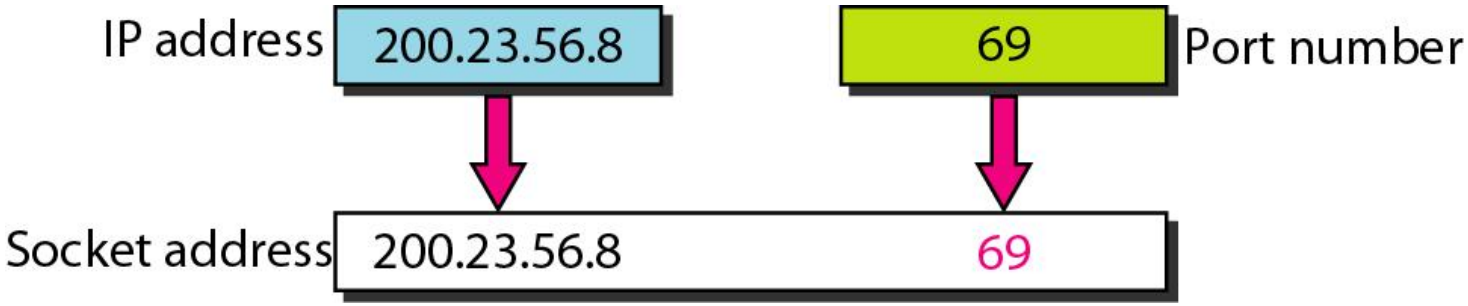
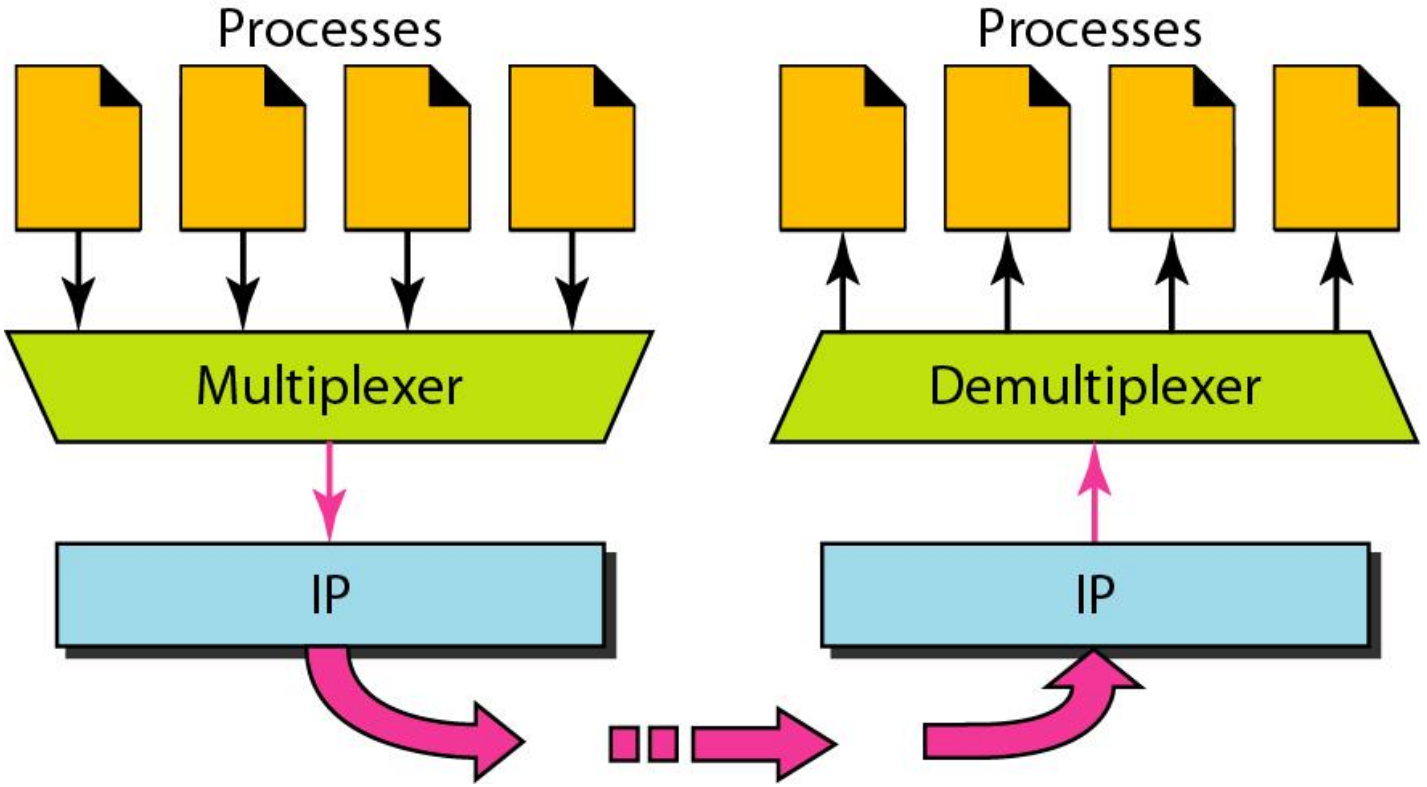
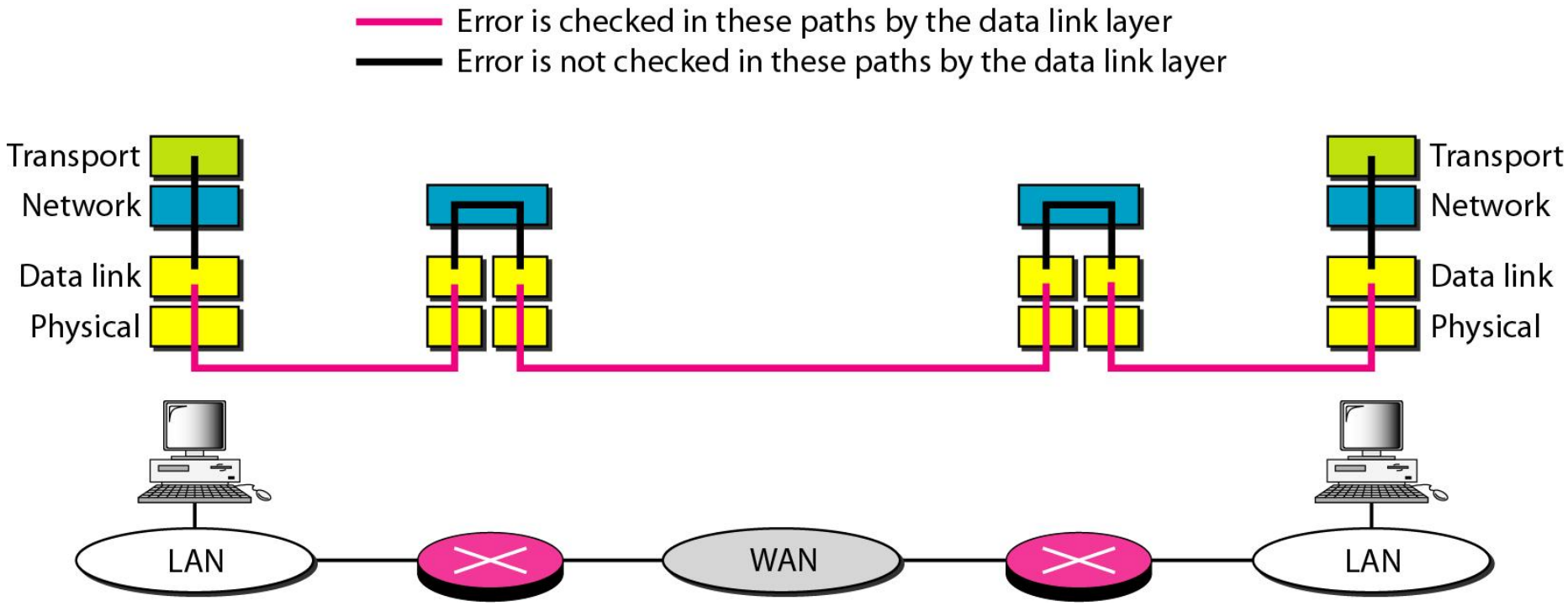


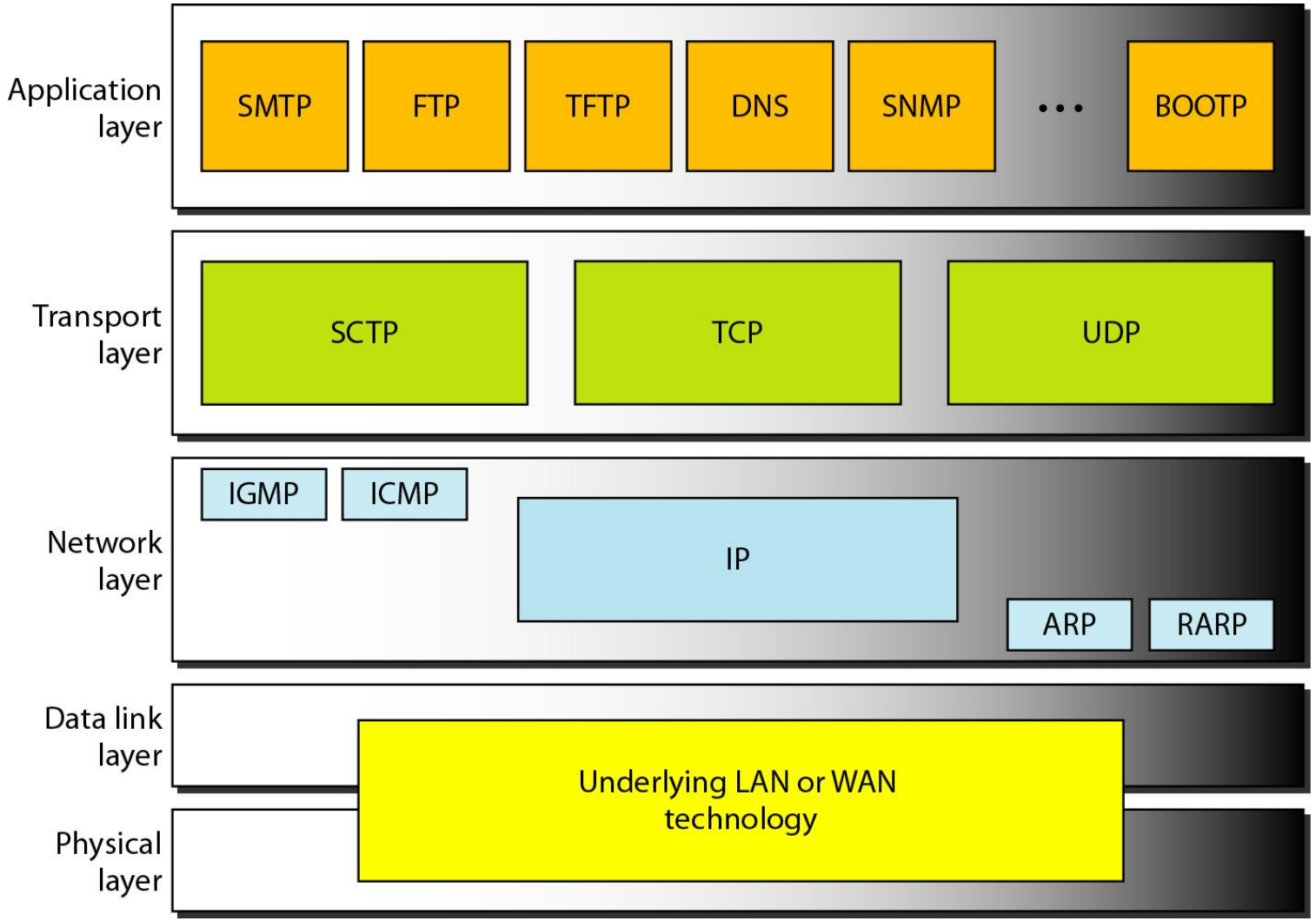
**Figure 6.6** *Multiplexing and demultiplexing (many to one relationship)*



## Figure 6.7 *Error control*



## Figure 6.8 *Position of UDP, TCP, and SCTP in TCP/IP suite*



# 6-2 USER DATAGRAM PROTOCOL (UDP)

*The User Datagram Protocol (UDP) is called a connectionless, unreliable transport protocol. It does not add anything to the services of IP except to provide process-to-process communication instead of host-to-host communication.*

***Topics discussed in this section:***

**Well-Known Ports for UDP**
**User Datagram**
**Checksum**
**UDP Operation**
**Use of UDP**

## Table 6.1 *Well-known ports used with UDP*

| *Port* | *Protocol* | *Description* |
|---|---|---|
| 7 | Echo | Echoes a received datagram back to the sender |
| 9 | Discard | Discards any datagram that is received |
| 11 | Users | Active users |
| 13 | Daytime | Returns the date and the time |
| 17 | Quote | Returns a quote of the day |
| 19 | Chargen | Returns a string of characters |
| 53 | Nameserver | Domain Name Service |
| 67 | BOOTPs | Server port to download bootstrap information |
| 68 | BOOTPc | Client port to download bootstrap information |
| 69 | TFTP | Trivial File Transfer Protocol |
| 111 | RPC | Remote Procedure Call |
| 123 | NTP | Network Time Protocol |
| 161 | SNMP | Simple Network Management Protocol |
| 162 | SNMP | Simple Network Management Protocol (trap) |

# Example 6.1

*In UNIX, the well-known ports are stored in a file called /etc/services. Each line in this file gives the name of the server and the well-known port number. We can use the grep utility to extract the line corresponding to the desired application. The following shows the port for FTP. Note that FTP can use port 21 with either UDP or TCP.*

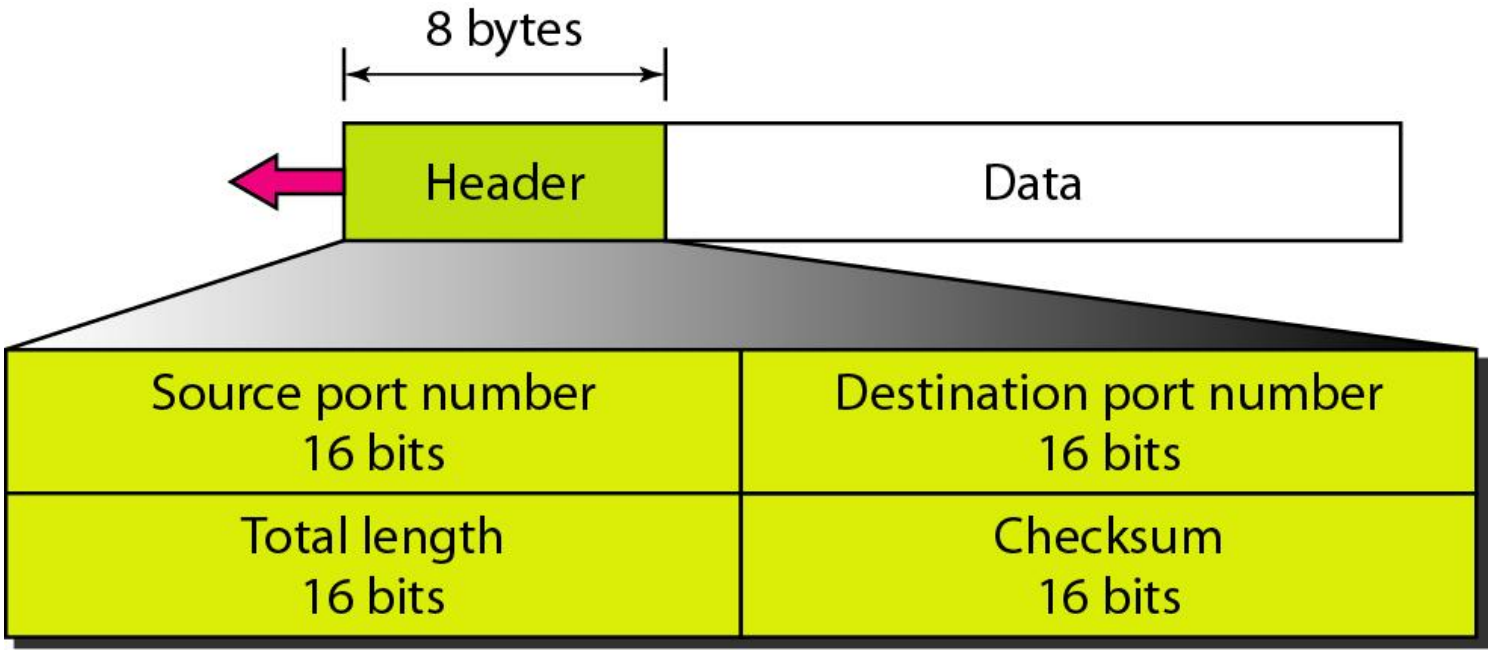
```
$ grep    ftp   /etc/services
ftp           21/tcp
ftp           21/udp
```

# Example 6.1 (continued)

***SNMP uses two port numbers (161 and 162), each for a different purpose.***

```
$ grep      snmp /etc/services
snmp            161/tcp         #Simple Net  Mgmt Proto
snmp            161/udp         #Simple Net  Mgmt Proto
snmptrap        162/udp         #Traps for SNMP
```

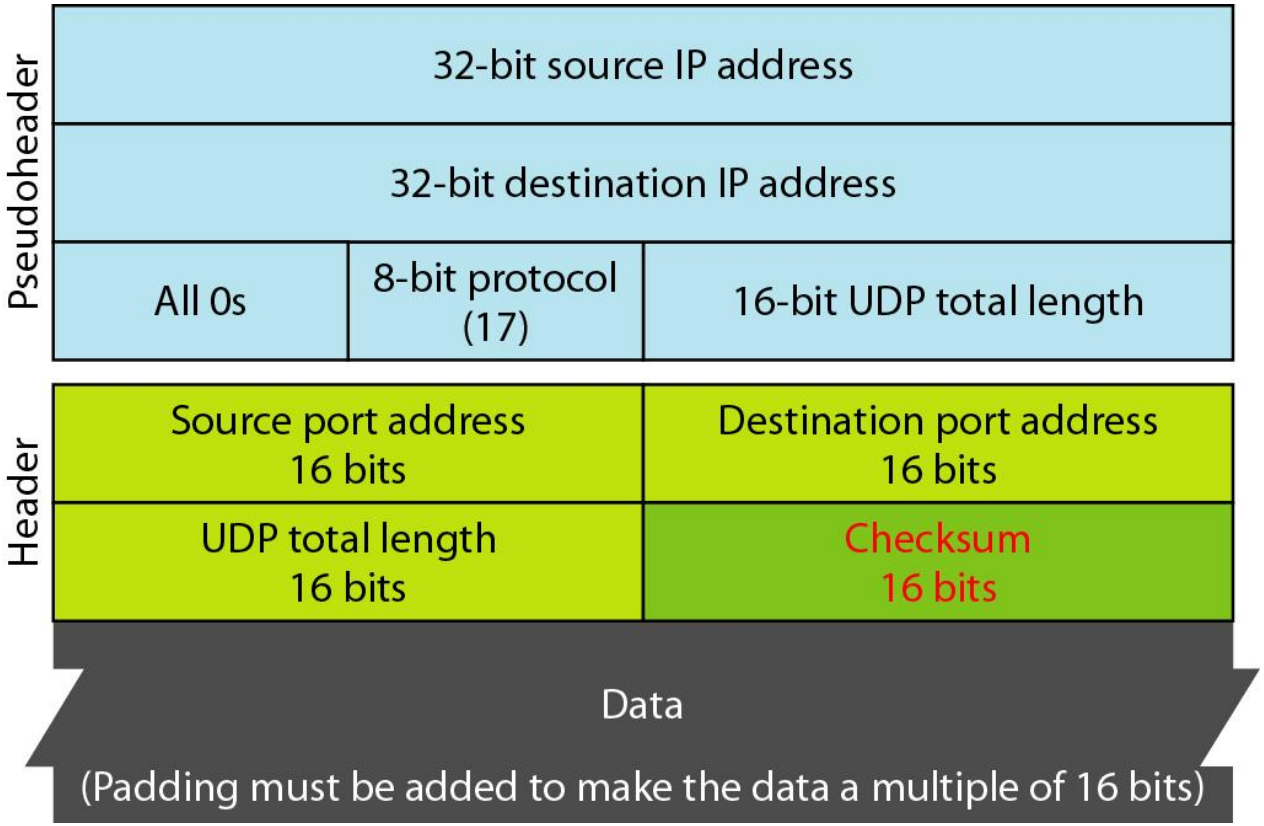
## Figure 6.9 *User datagram format*



*Note*

**UDP length**
**= IP length – IP header's length**

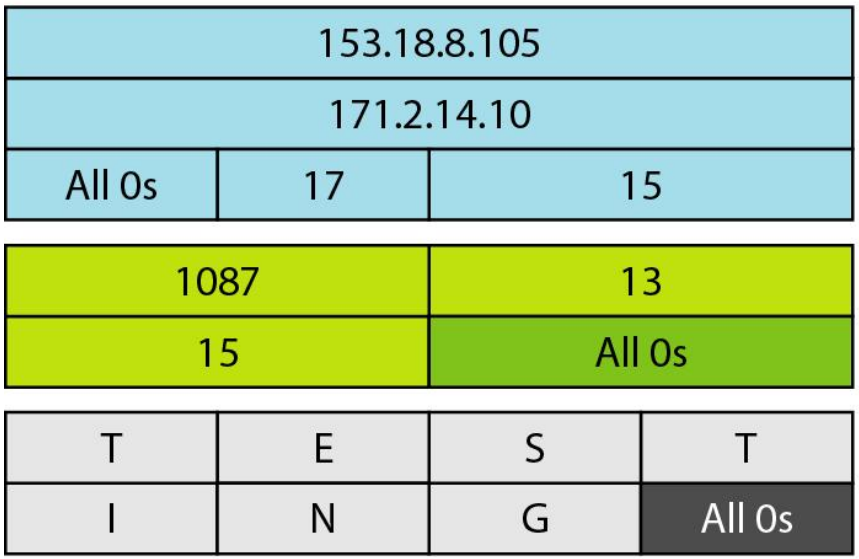
## Figure 6.10 *Pseudoheader for checksum calculation*



# Example 6.2

*Figure 6.11 shows the checksum calculation for a very small user datagram with only 7 bytes of data. Because the number of bytes of data is odd, padding is added for checksum calculation. The pseudoheader as well as the padding will be dropped when the user datagram is delivered to IP.*

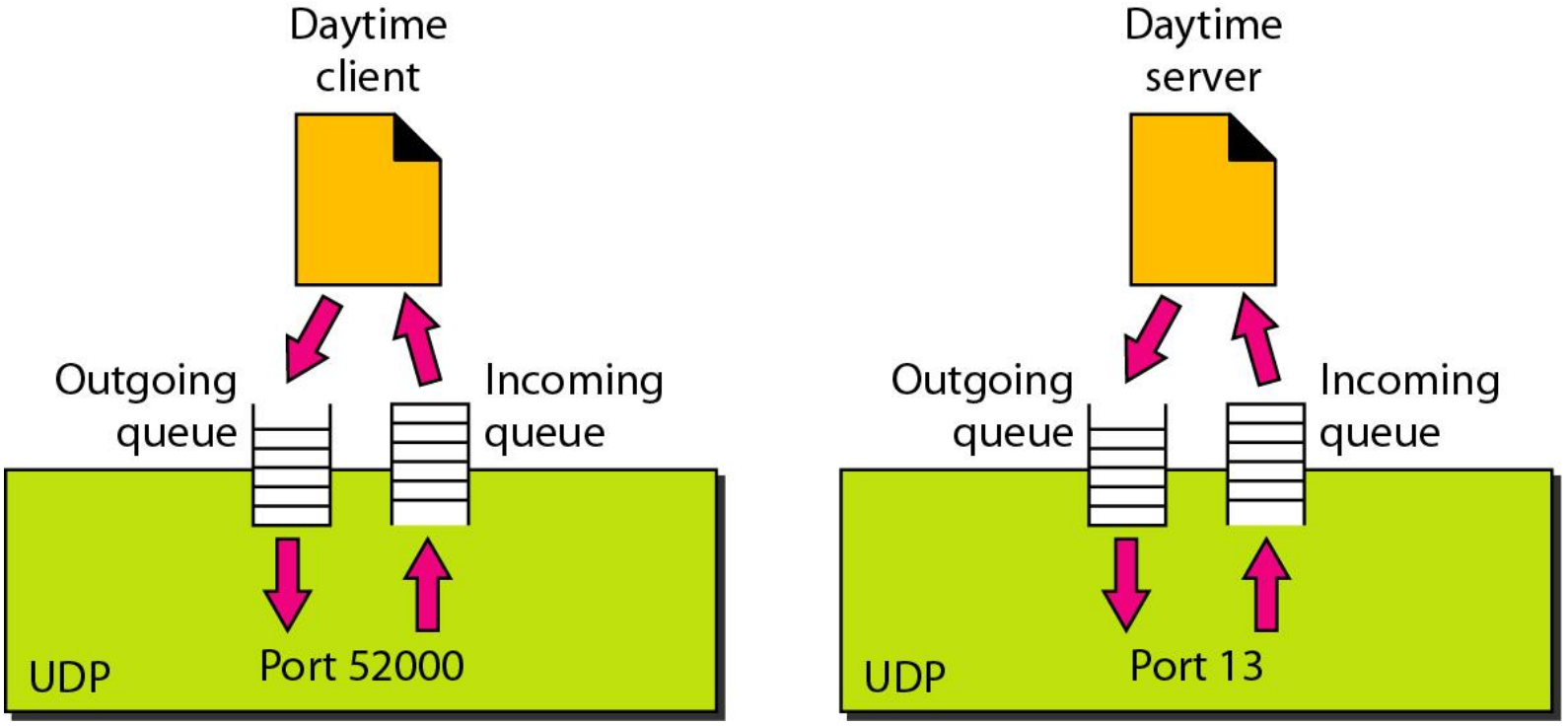
## Figure 6.11 *Checksum calculation of a simple UDP user datagram*




10011001 00010010 → 153.18
00001000 01101001 → 8.105
10101011 00000010 → 171.2
00001110 00001010 → 14.10
00000000 00010001 → 0 and 17
00000000 00001111 → 15
00000100 00111111 → 1087
00000000 00001101 → 13
00000000 00001111 → 15
00000000 00000000 → 0 (checksum)
01010100 01000101 → T and E
01010011 01010100 → S and T
01001001 01001110 → I and N
01000111 00000000 → G and 0 (padding)

10010110 11101011 → Sum
01101001 00010100 → Checksum

## Figure 6.12 *Queues in UDP*



# 6-3 TCP

*TCP is a connection-oriented protocol; it creates a virtual connection between two TCPs to send data. In addition, TCP uses flow and error control mechanisms at the transport level.*

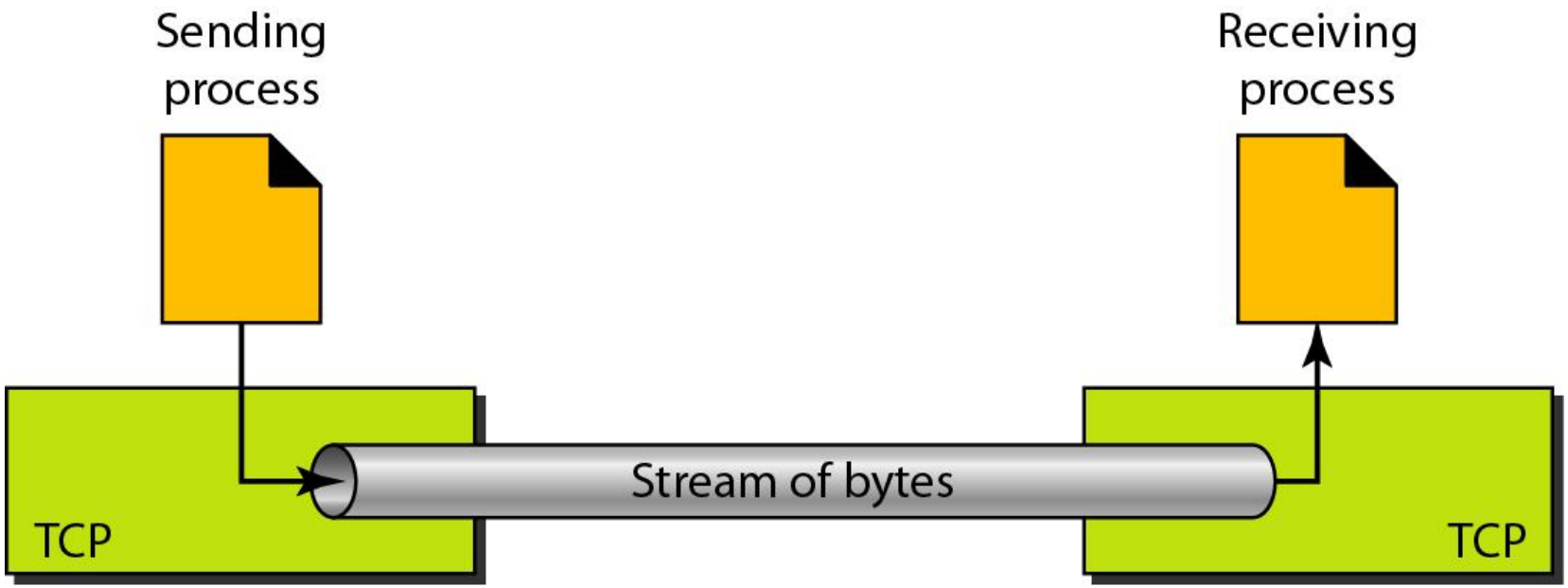
***Topics discussed in this section:***

**TCP Services**
**TCP Features**
**Segment**
**A TCP Connection**
**Flow Control**
**Error Control**

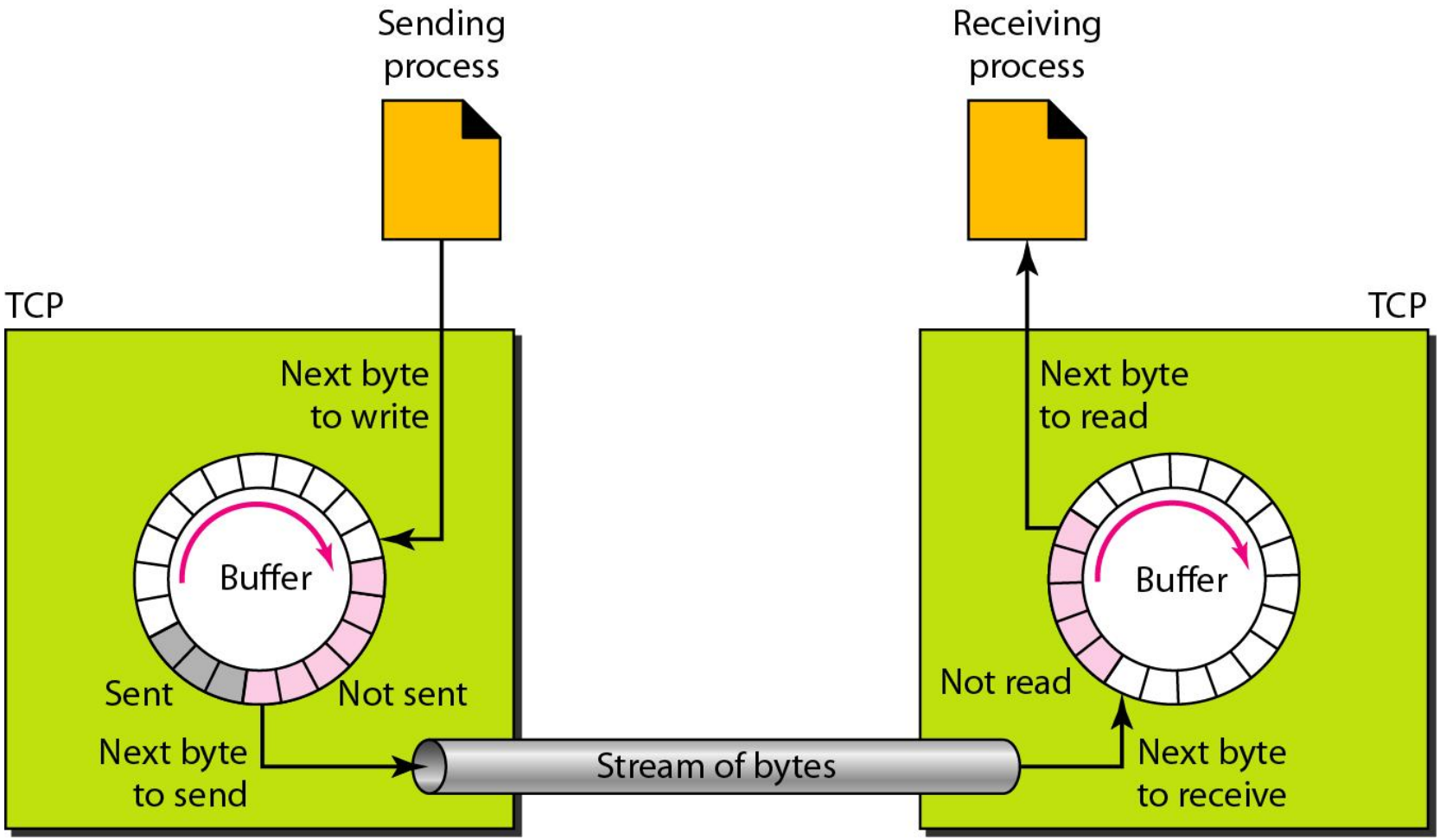
## Table 6.2 *Well-known ports used by TCP*

| *Port* | *Protocol* | *Description* |
|---|---|---|
| 7 | Echo | Echoes a received datagram back to the sender |
| 9 | Discard | Discards any datagram that is received |
| 11 | Users | Active users |
| 13 | Daytime | Returns the date and the time |
| 17 | Quote | Returns a quote of the day |
| 19 | Chargen | Returns a string of characters |
| 20 | FTP, Data | File Transfer Protocol (data connection) |
| 21 | FTP, Control | File Transfer Protocol (control connection) |
| 23 | TELNET | Terminal Network |
| 25 | SMTP | Simple Mail Transfer Protocol |
| 53 | DNS | Domain Name Server |
| 67 | BOOTP | Bootstrap Protocol |
| 79 | Finger | Finger |
| 80 | HTTP | Hypertext Transfer Protocol |
| 111 | RPC | Remote Procedure Call |

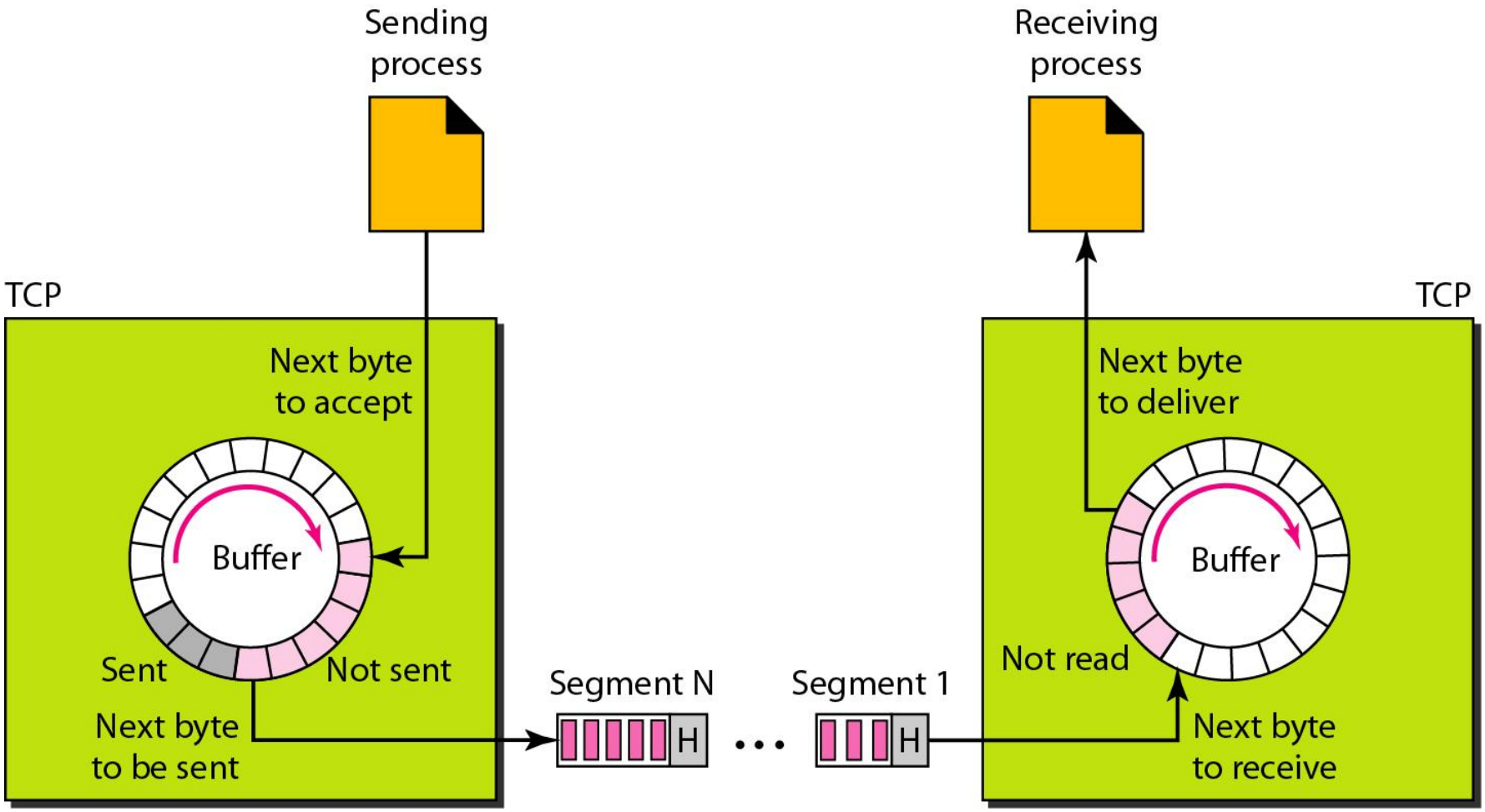
## Figure 6.13 *Stream delivery*



## Figure 6.14 *Sending and receiving buffers*



## Figure 6.15 *TCP segments*



*Note*

**The bytes of data being transferred in each connection are numbered by TCP. The numbering starts with a randomly generated number.**

# Example 6.3

*The following shows the sequence number for each segment:*

| | | |
|---|---|---|
| Segment 1 | → | Sequence Number: 10,001 (range: 10,001 to 11,000) |
| Segment 2 | → | Sequence Number: 11,001 (range: 11,001 to 12,000) |
| Segment 3 | → | Sequence Number: 12,001 (range: 12,001 to 13,000) |
| Segment 4 | → | Sequence Number: 13,001 (range: 13,001 to 14,000) |
| Segment 5 | → | Sequence Number: 14,001 (range: 14,001 to 15,000) |

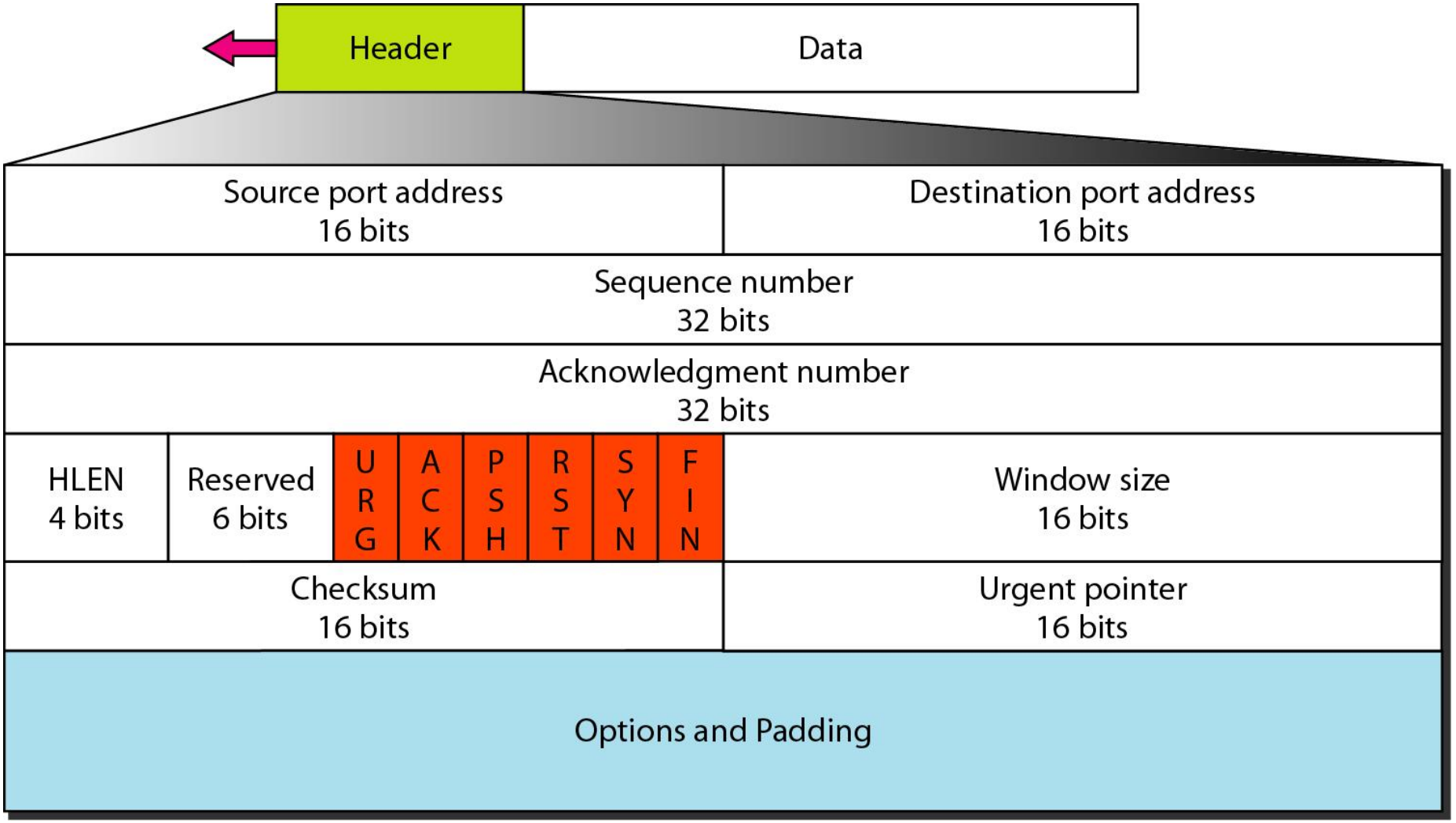
*Note*

**The value in the sequence number field of a segment defines the number of the first data byte contained in that segment.**

*Note*

**The value of the acknowledgment field in a segment defines the number of the next byte a party expects to receive. The acknowledgment number is cumulative.**

## Figure 6.16 *TCP segment format*



## Figure 6.17 *Control field*

URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push

RST: Reset the connection
SYN: Synchronize sequence numbers
FIN: Terminate the connection

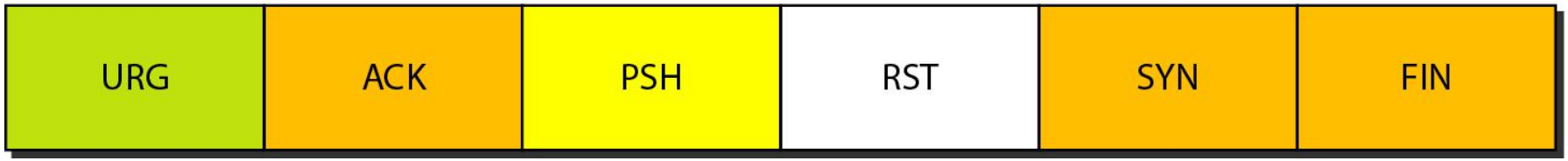
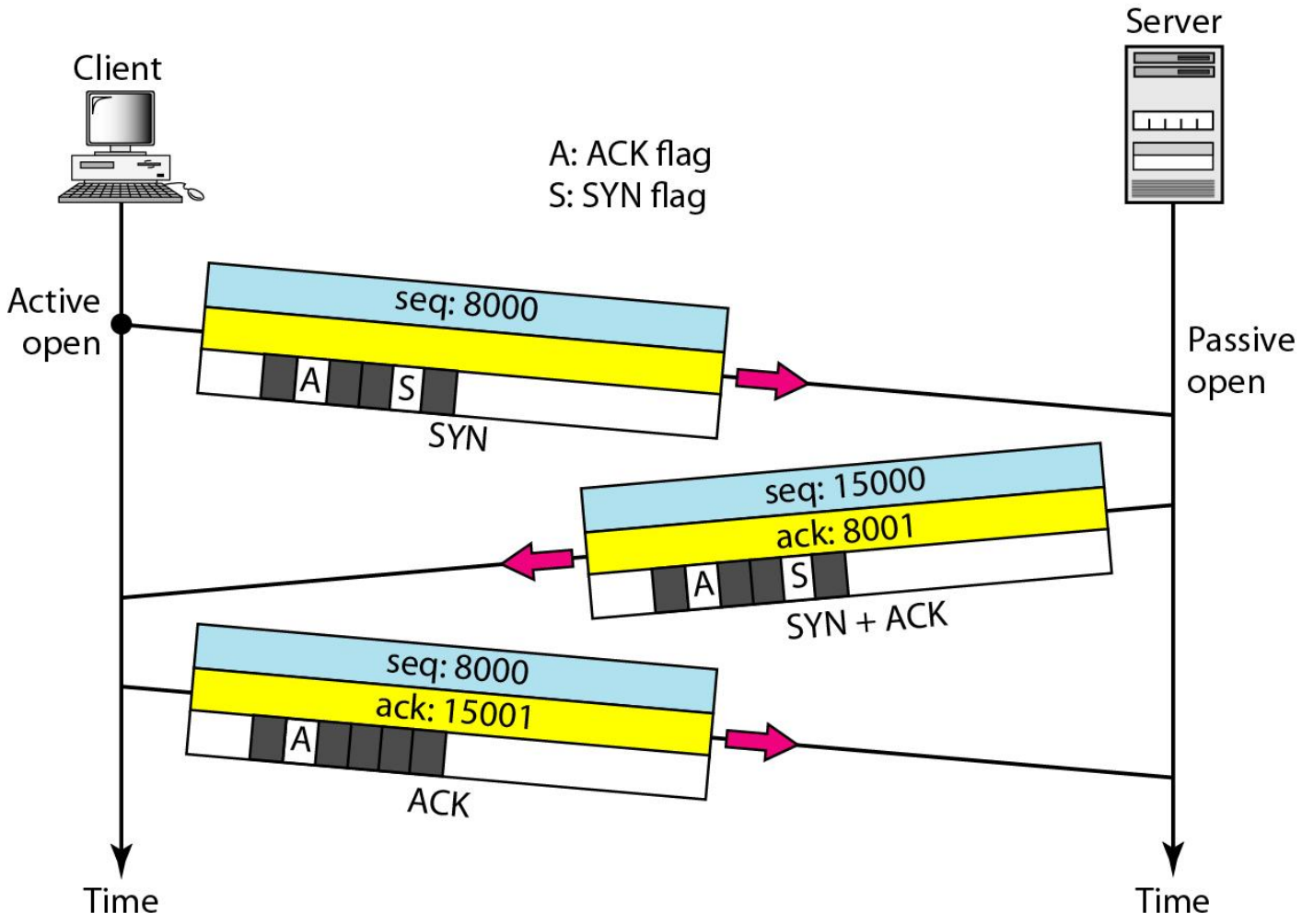


**Table 6.3** ***Description of flags in the control field***

| *Flag* | *Description* |
|---|---|
| URG | The value of the urgent pointer field is valid. |
| ACK | The value of the acknowledgment field is valid. |
| PSH | Push the data. |
| RST | Reset the connection. |
| SYN | Synchronize sequence numbers during connection. |
| FIN | Terminate the connection. |

## Figure 6.18 *Connection establishment using three-way handshaking*



*Note*

**A SYN segment cannot carry data, but it consumes one sequence number.**

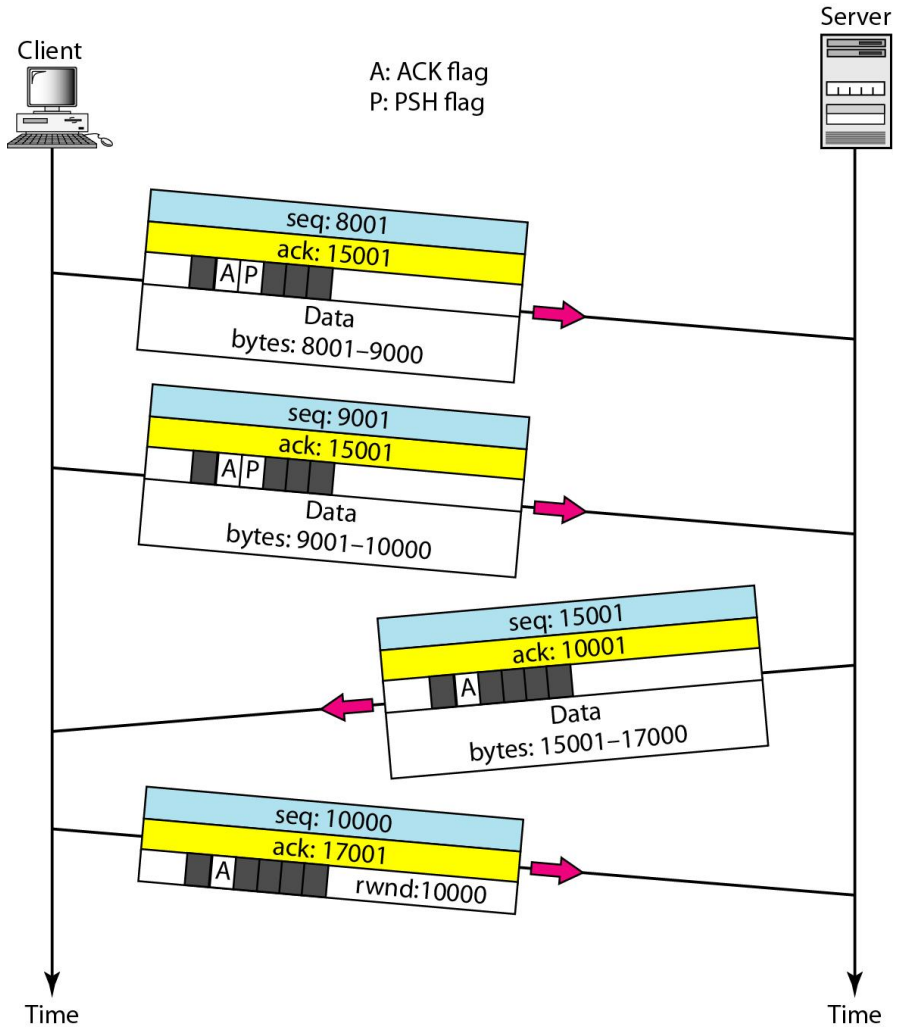
*Note*

**A SYN + ACK segment cannot carry data, but does consume one sequence number.**

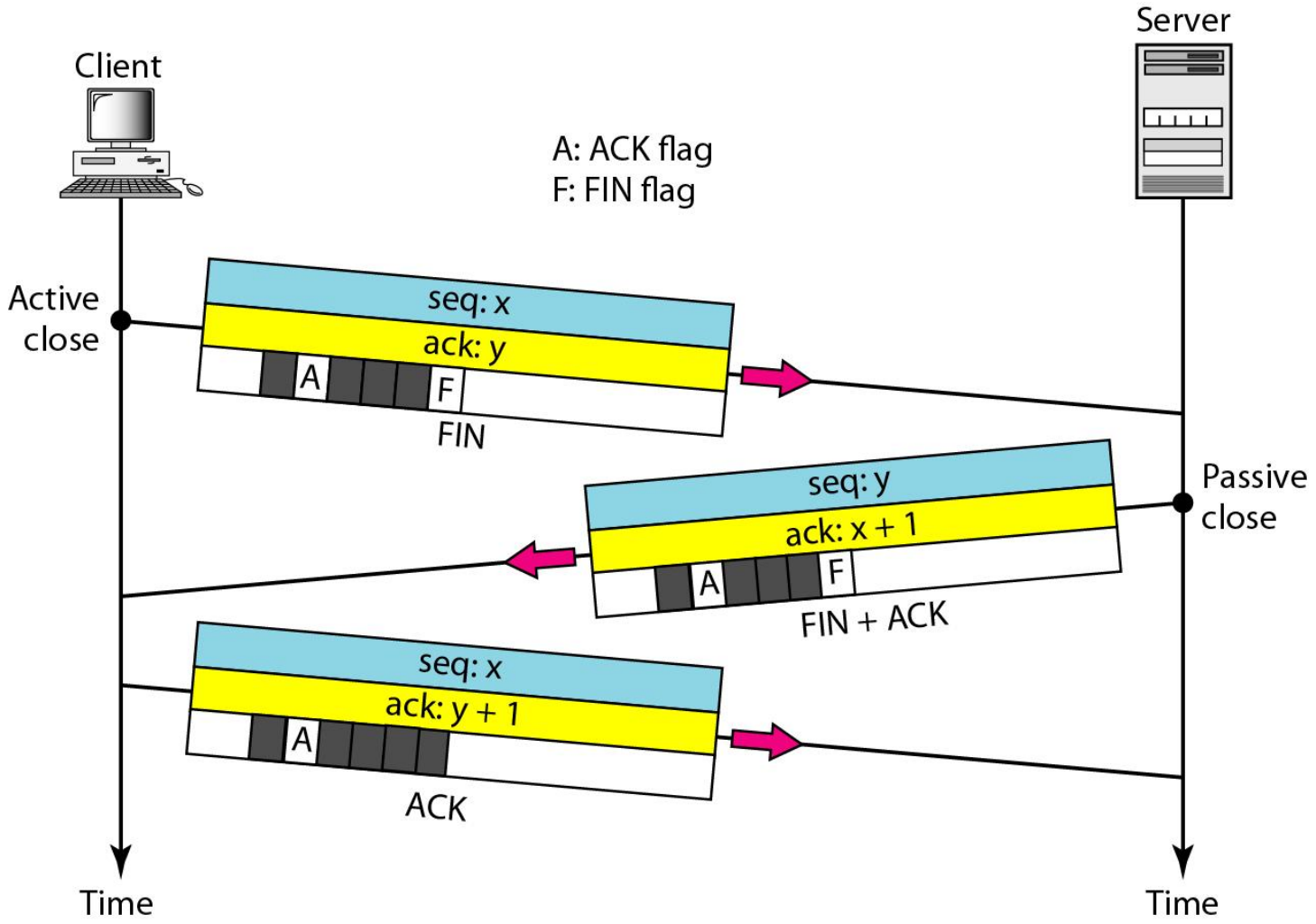
*Note*

**An ACK segment, if carrying no data, consumes no sequence number.**

# Figure 6.19 *Data transfer*



## Figure 6.20 *Connection termination using three-way handshaking*



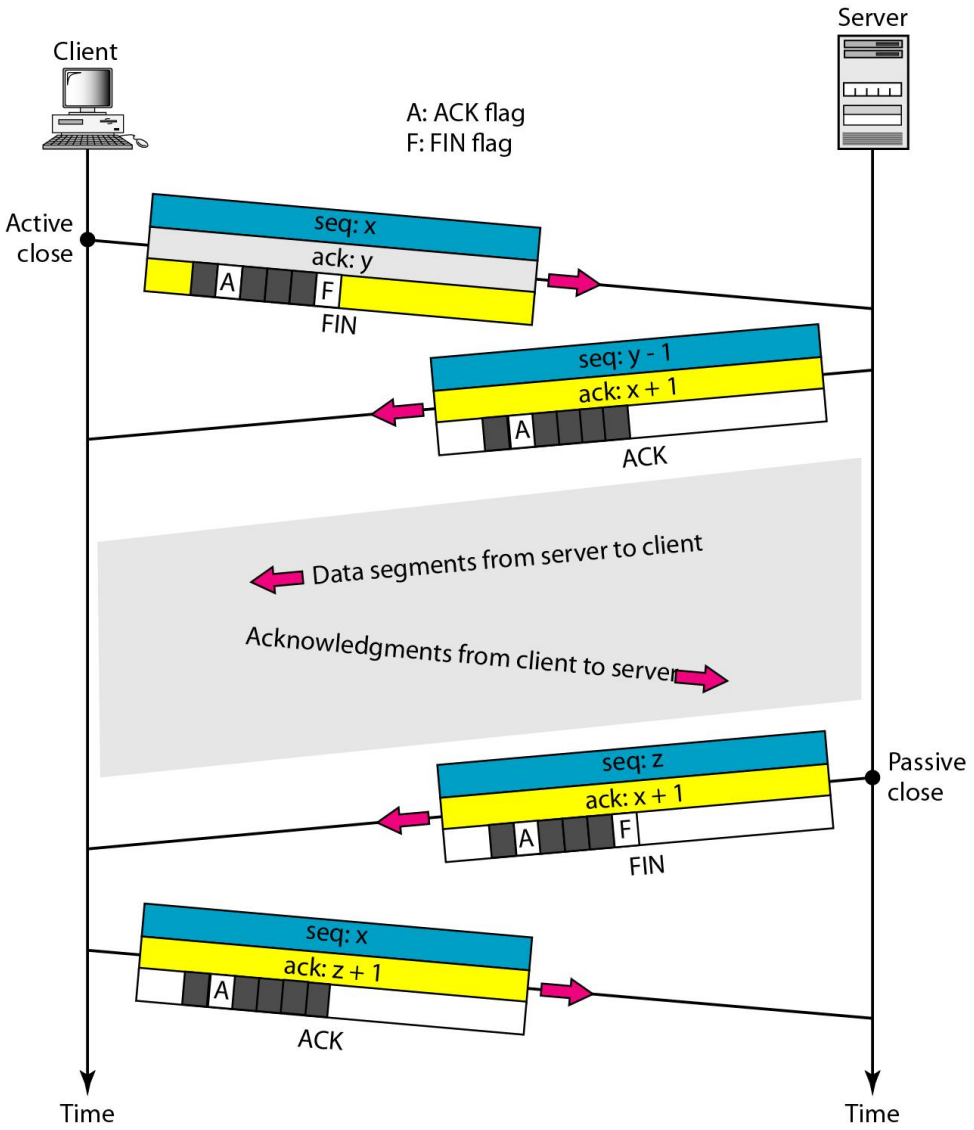
*Note*

**The FIN segment consumes one sequence number if it does not carry data.**

*Note*

**The FIN + ACK segment consumes one sequence number if it does not carry data.**

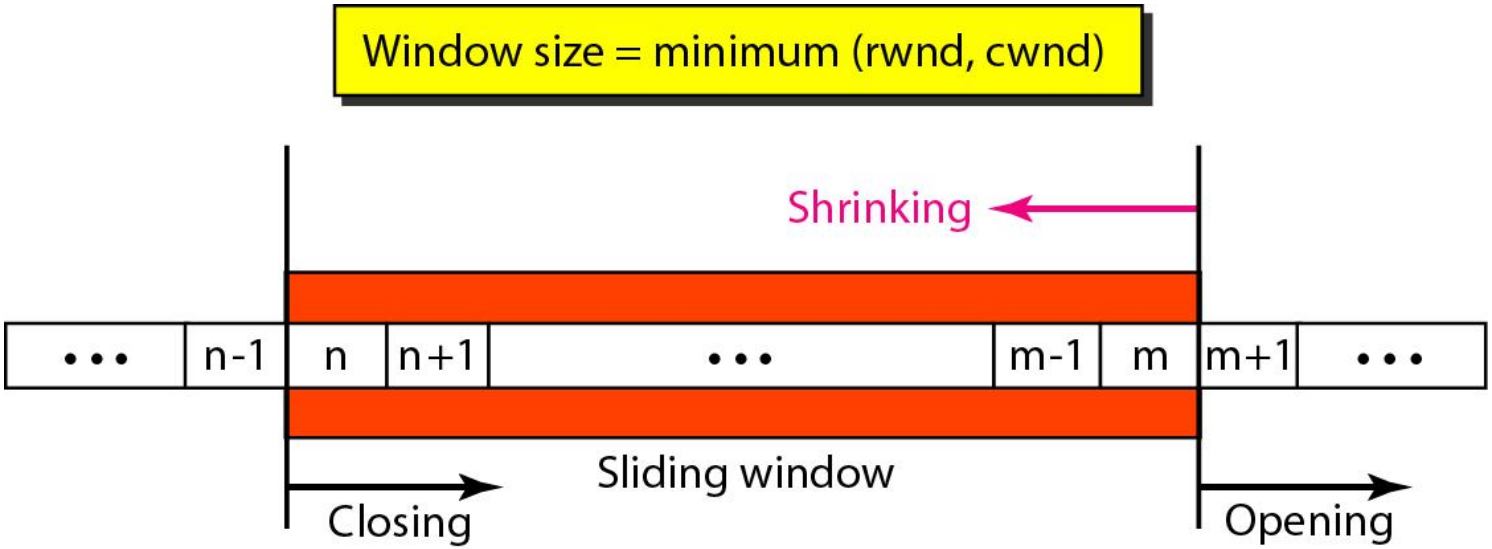
# Figure 6.21 *Half-close*




In TCP, one end can stop sending data while still receiving data. This is called a half-close.

## Figure 6.22 *Sliding window (Three activities)*

Receiver window (rwnd): is the value advertised by the opposite end in a segment containing ack. It is the number of bytes the other end can accepted before its buffer overflows and data are discarded.
Congestion window (cwnd): is a value determined by the network to avoid congestion.




**Opening:** allows more new bytes in the buffer that are eligible for sending.
**Closing:** some bytes have been acked and the sender needs not worry about them any more.
**Shrinking:** means revoking the eligibility of some bytes for sending. This is a problem if the sender has already sent these bytes. This is strongly discouraged and not allowed in some implementations.

*Note*

**A sliding window is used to make transmission more efficient as well as to control the flow of data so that the destination does not become overwhelmed with data.**
**TCP sliding windows are byte-oriented.**

# Example 6.4

*What is the value of the receiver window (rwnd) for host A if the receiver, host B, has a buffer size of 5000 bytes and 1000 bytes of received and unprocessed data?*

## Solution

*The value of rwnd = 5000 − 1000 = 4000. Host B can receive only 4000 bytes of data before overflowing its buffer. Host B advertises this value in its next segment to A.*

# Example 6.5

*What is the size of the window for host A if the value of rwnd is 3000 bytes and the value of cwnd is 3500 bytes?*

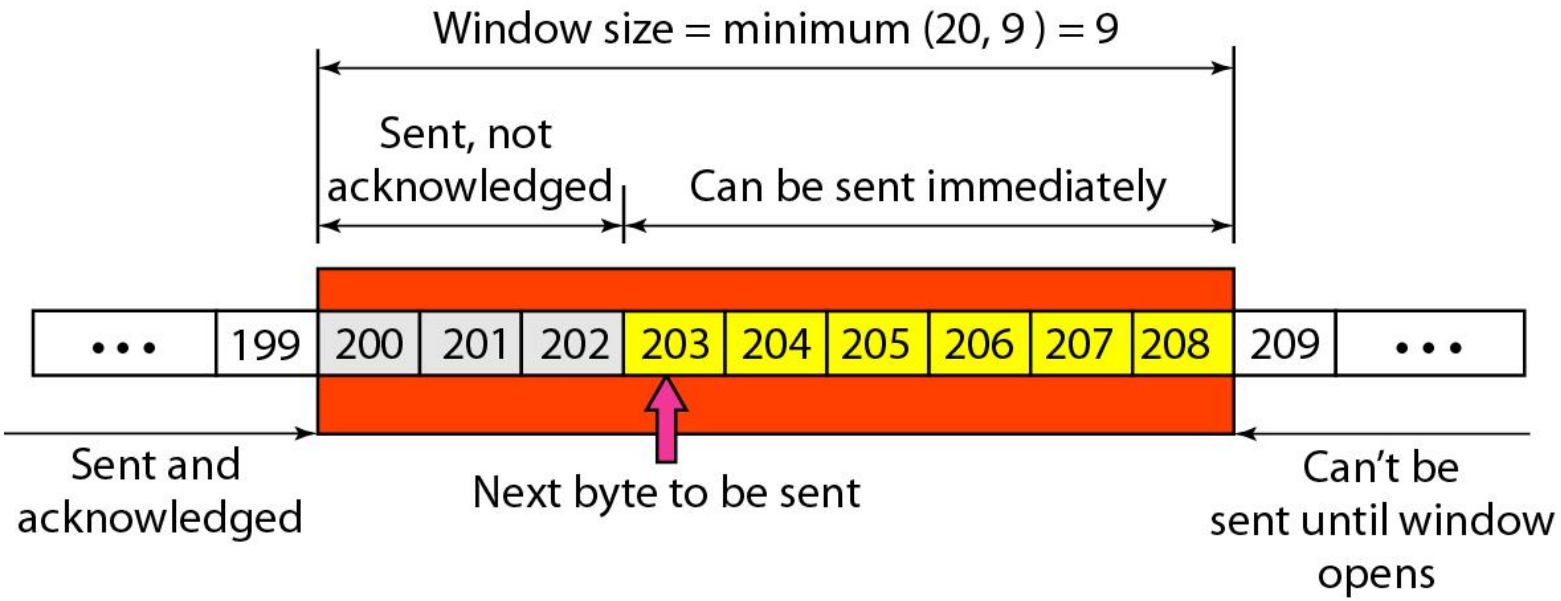
*Solution*

*The size of the window is the smaller of rwnd and cwnd, which is 3000 bytes.*

# Example 6.6

*Figure 6.23 shows an unrealistic example of a sliding window. The sender has sent bytes up to 202. We assume that cwnd is 20 (in reality this value is thousands of bytes). The receiver has sent an acknowledgment number of 200 with an rwnd of 9 bytes (in reality this value is thousands of bytes). The size of the sender window is the minimum of rwnd and cwnd, or 9 bytes. Bytes 200 to 202 are sent, but not acknowledged. Bytes 203 to 208 can be sent without worrying about acknowledgment. Bytes 209 and above cannot be sent.*

## Figure 6.23 *Example 6.6*



Note

## Some points about TCP sliding windows:

- The size of the window is the lesser of rwnd and cwnd.
- The source does not have to send a full window's worth of data.
- The window can be opened or closed by the receiver, but should not be shrunk.
- The destination can send an acknowledgment at any time as long as it does not result in a shrinking window.
- The receiver can temporarily shut down the window; the sender, however, can always send a segment of 1 byte after the window is shut down.

*Note*

**ACK segments do not consume sequence numbers and are not acknowledged.**

*Note*

**In modern implementations, a retransmission occurs if the retransmission timer expires or three duplicate ACK segments have arrived.**

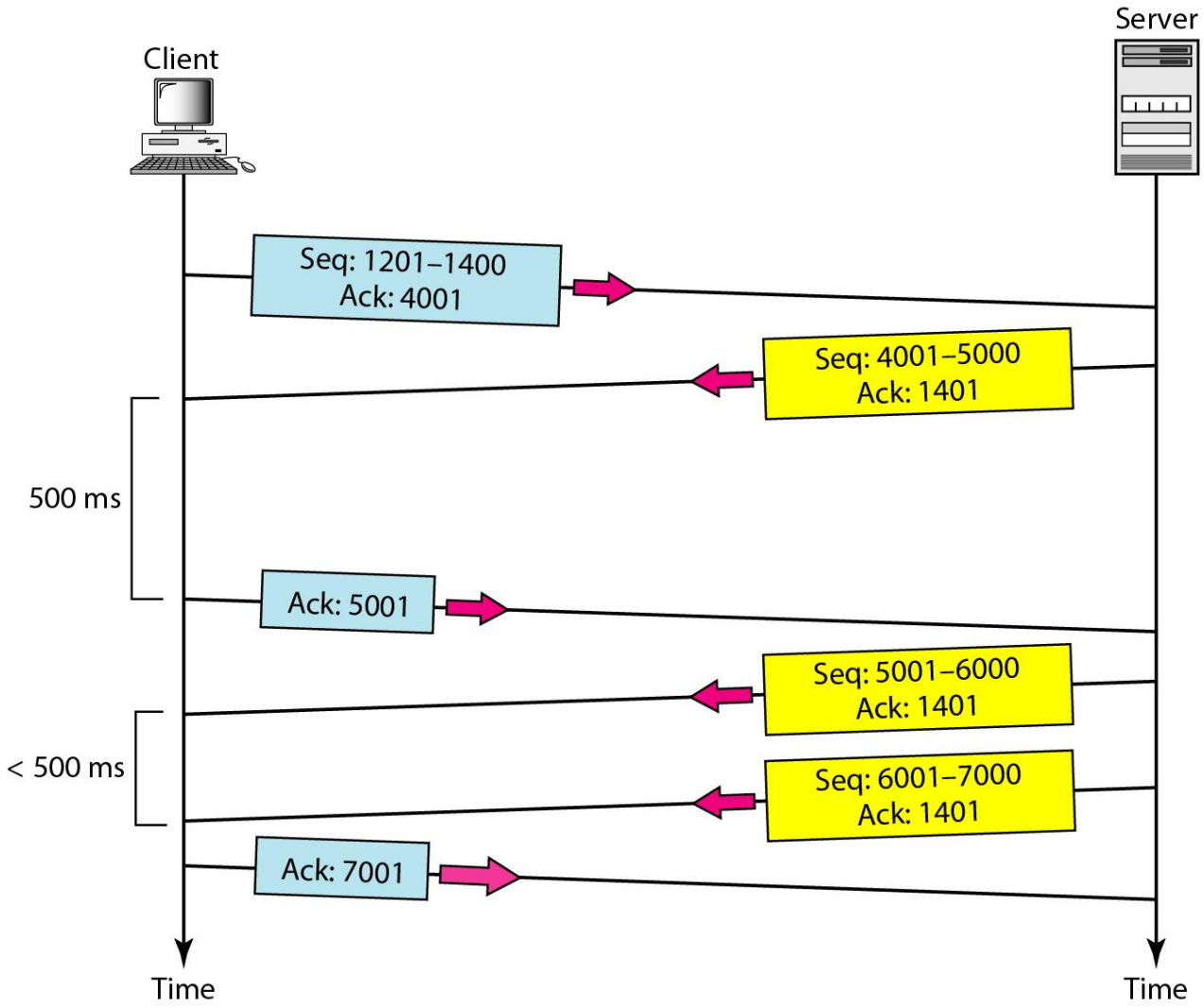
*Note*

**No retransmission timer is set for an ACK segment.**

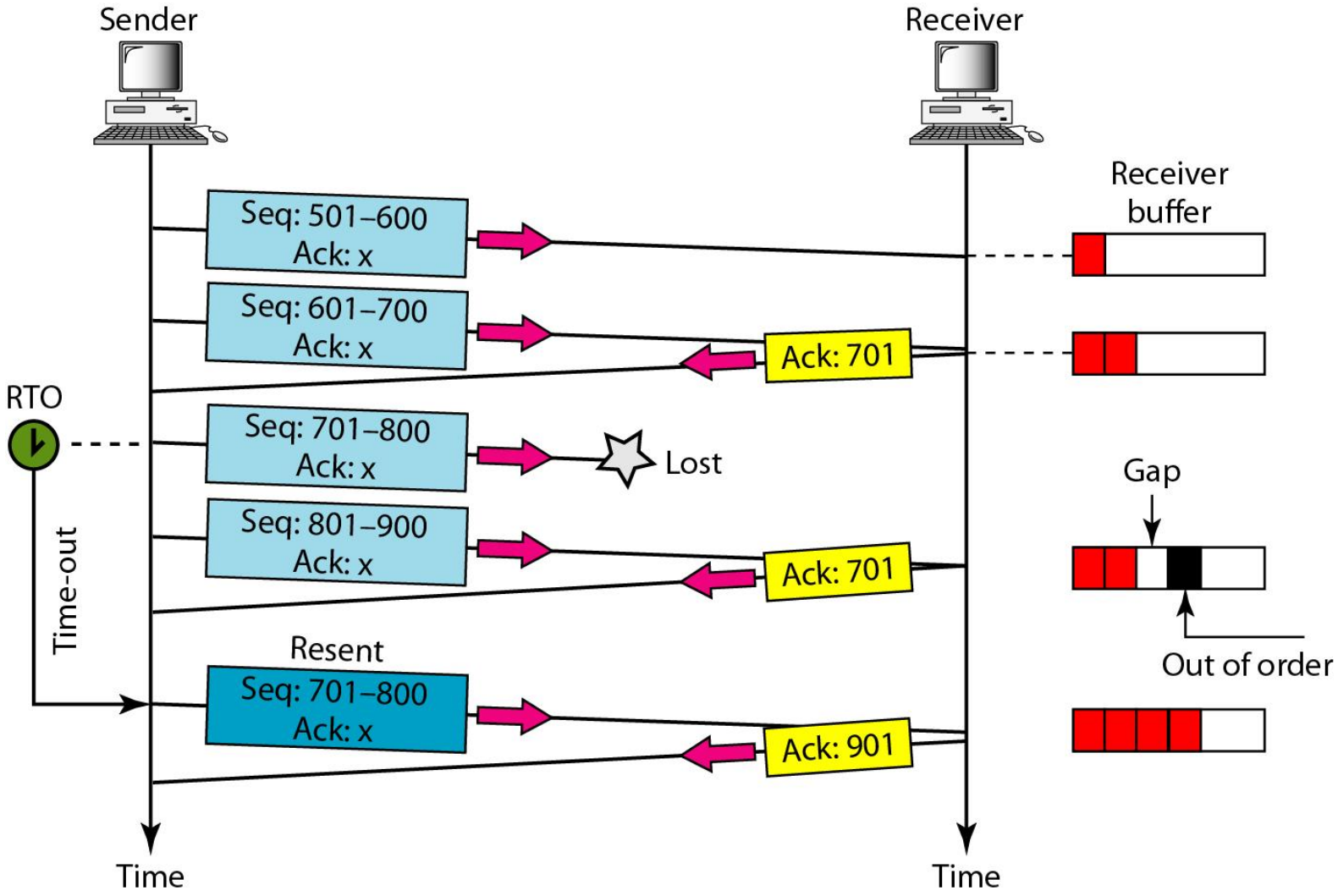
*Note*

**Data may arrive out of order and be temporarily stored by the receiving TCP, but TCP guarantees that no out-of-order segment is delivered to the process.**

# Figure 6.24 *Normal operation*



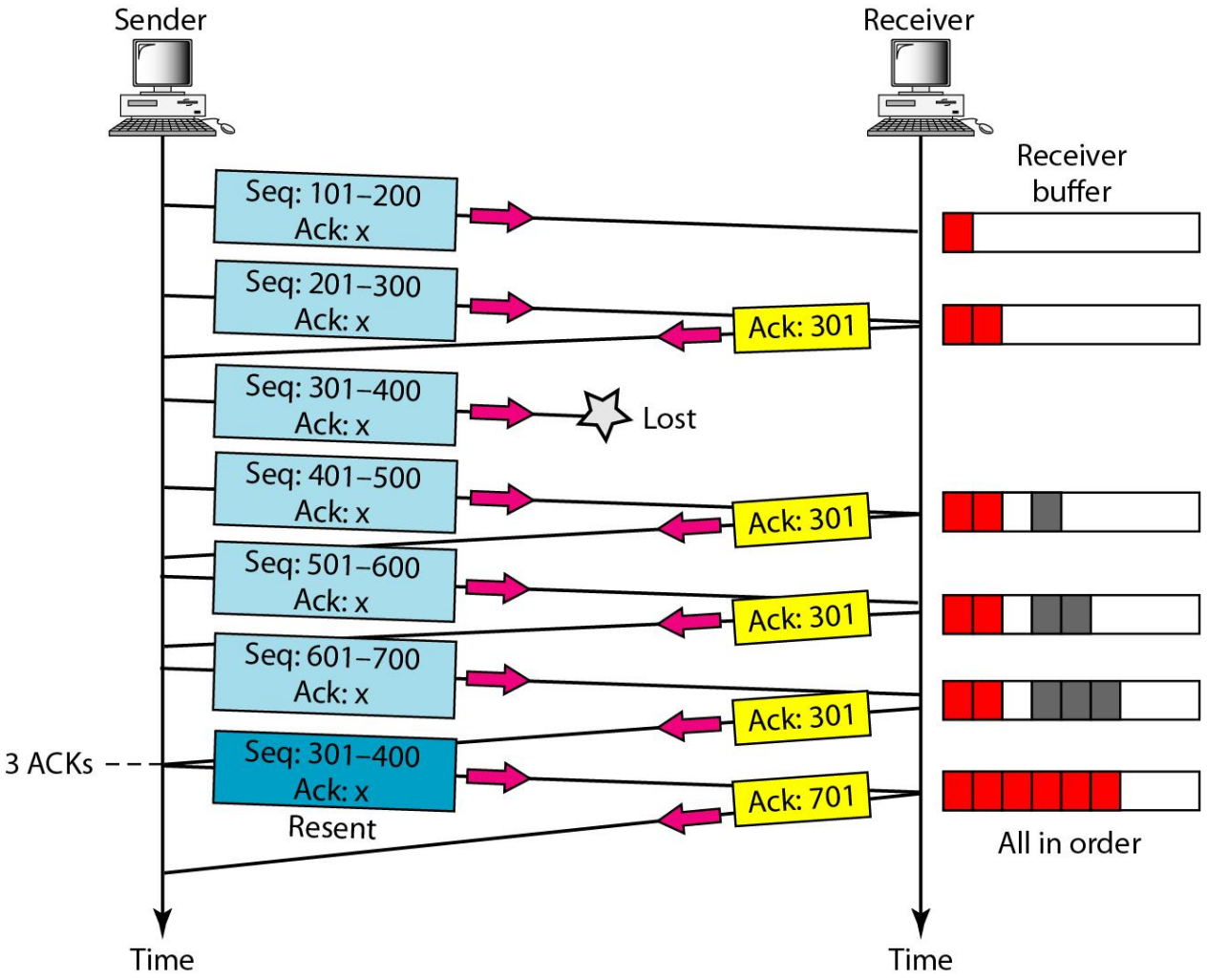
## Figure 6.25 *Lost segment*



*Note*

**The receiver TCP delivers only ordered data to the process.**

# Figure 6.26 *Fast retransmission*



# Summary

- In the client-server model, the client runs a program to request a service and the server runs a program to provide the service. These two programs communicate with each other.
- One server program can provide services for many client programs.
- Clients can be run either iteratively (one at a time) or concurrently (many at a time).
- Servers can handle clients either iteratively (one at a time) or concurrently (many at a time).
- A connectionless iterative server uses UDP as its transport layer protocol and can serve one client at a time.

# Summary-Cont.

- A connection-oriented concurrent server uses TCP as its transport layer protocol and can serve many clients at the same time.
- When the operating system executes a program, an instance of the program, called a process, is created.
- If two application programs, one running on a local system and the other running on the remote system, need to communicate with each other, a network program is required.
- The socket interface is a set of declarations, definitions, and procedures for writing cleint-server programs.
- The communication structure needed for socket programming is called a socket.
- A stream socket is used with a connection-oriented protocol such as TCP.
- A datagram socket is used with a connectionless protocol such as UDP.